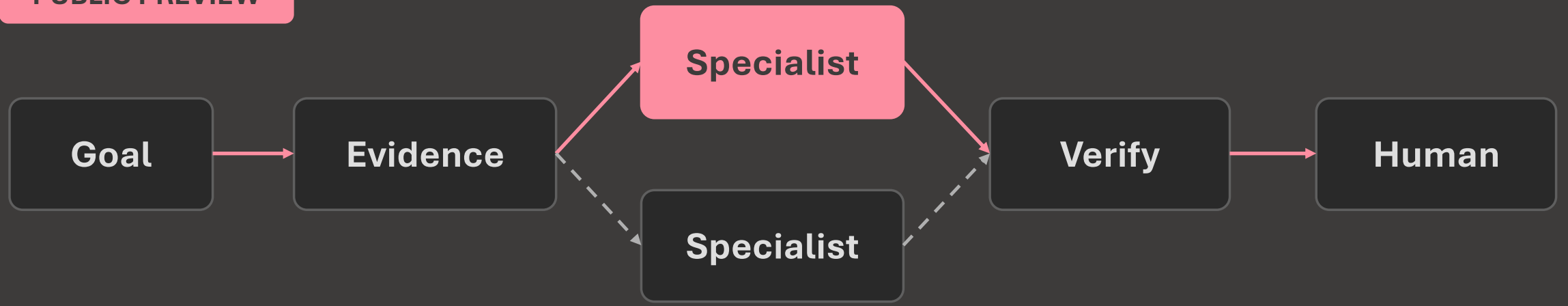
Don't just prompt. Orchestrate.

Agentic work + dynamic workflows
with GitHub Copilot

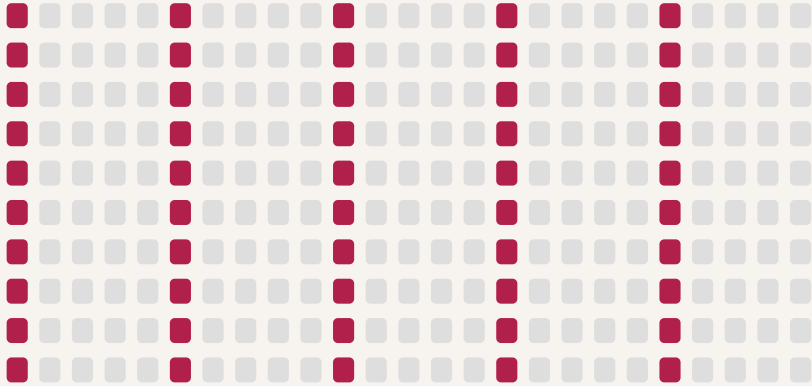
PUBLIC PREVIEW



2,500 people.
One broken launch.

2,500

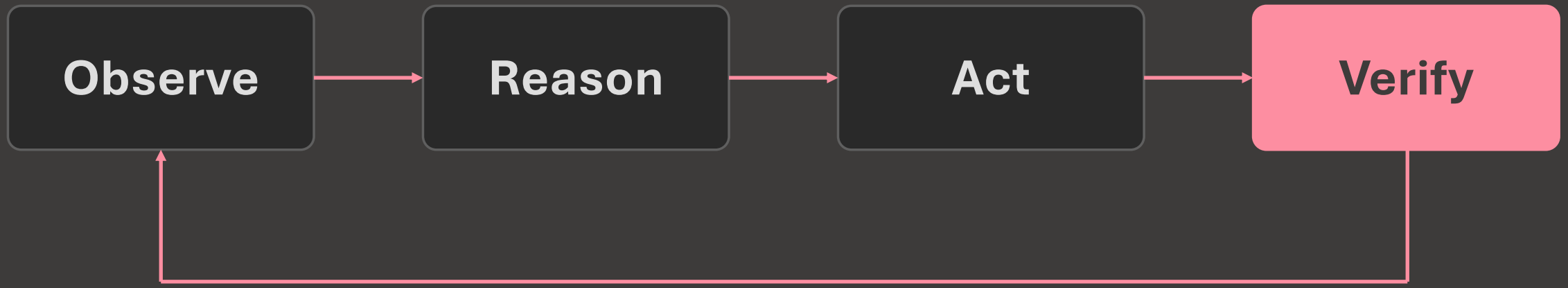
synthetic attendees



Each dot = 10 attendees.
Every fifth attendee retries.

What happens when the first plan becomes wrong?

An agent needs a feedback loop.



Autonomy controls when it stops.
Verification determines whether it worked.

Three controls.
Three different jobs.

Autopilot

Keep working.

The agent continues toward the goal.

/fleet

Delegate the work.

Copilot chooses the decomposition.

Dynamic workflow

Define the process.

Code owns the stages, branches, and handoffs.

Autonomy ≠ parallelism ≠ a reusable process

Same word. Different runtimes.

Copilot dynamic workflows

Extension program

CLI · app · SDK
Steps, agents, checkpoints, resume

THIS DEMO

GitHub Agentic Workflows

Markdown + frontmatter

Compile → GitHub Actions
Triggers, runner jobs, safe outputs

Two features.
One controlled process.

Dynamic Workflow

Code picks specialists.
Code branches on failed checks.
Code saves and pauses.

Agentic Workflow

Model interprets evidence.
Model explains or rejects.
Model never rewrites code.

Code owns the process. Models handle judgment.

Specialists need contracts, not just job titles.

A constrained specialist

```
model: gpt-6.1-sol
tools: []

Workflow call:
reasoningEffort: "max"
```

A bounded result

```
{
  repair: "dedupe",
  rationale: "...",
  evidenceIds: ["E-RETRY"]
}
```

Validate meaning, not just shape.

Reuse the rules.
Let the path change.

RUN A / retry evidence only



RUN B / both faults known up front



Identify the feature at every stop.

Dynamic Workflow

CODE chooses the steps.
Branches when checks fail.
Keeps completed work.

Agentic Workflow

MODEL interprets evidence.
Explains or rejects the repair.

Actual model handoff

Actual Copilot model result: gpt-6.1-sol / max

Relevant native code / excerpt

```
const result = await ctx.agent(prompt, {
  model: "gpt-6.1-sol", reasoningEffort: "max",
  schema: proposalSchema(repair)
});
```

Evidence in: 500 retry requests

```
repair: dedupe
cites: E-RETRY
```

The room picks the incident.

Retry storm

500 extra requests.
Identity must survive retries.

Room switch

The published map is stale.
Capacity must stay honest.

Both at once

Two independent findings.
Two specialists in parallel.

Surprise me

Plan first.
Then change the evidence.

One command.
A real Copilot workflow.

```
node .\native\Run-Sdk.mjs --live --pause --walkthrough
```

Analysis

GPT-6.1 Sol / max

Independent challenge

Sonnet 5.5 / high

5 teaching stops: NEXT. Final draft review: APPROVE.

The surprise is new evidence, not a new prompt.

1,800 → **1,600**

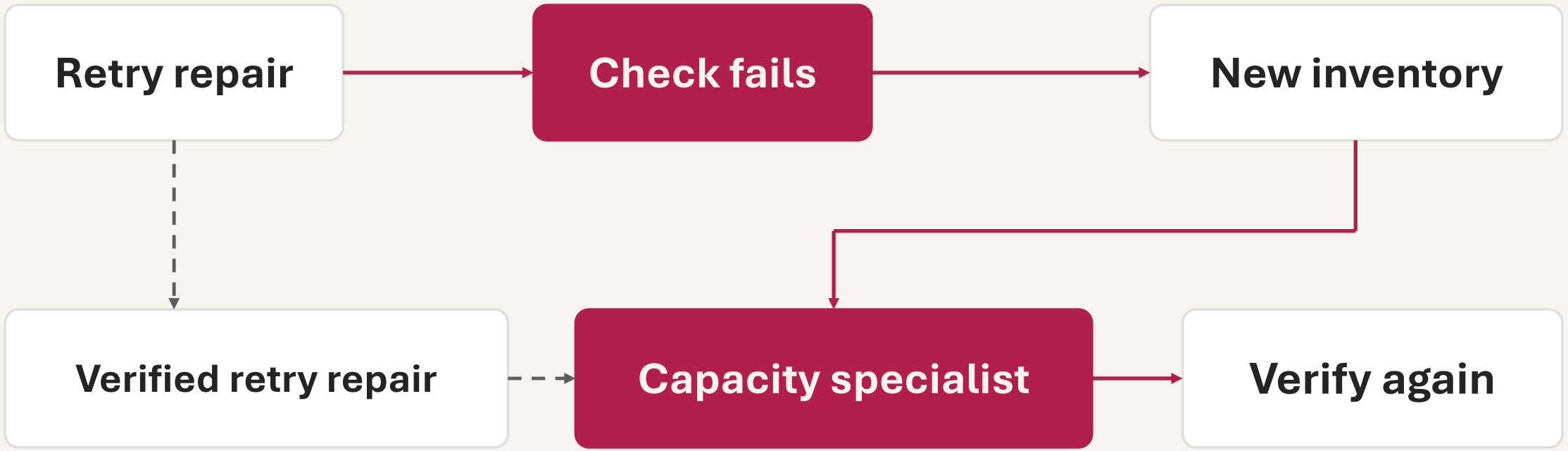
planned main hall seats

current main hall seats

C-CAPACITY: FAIL

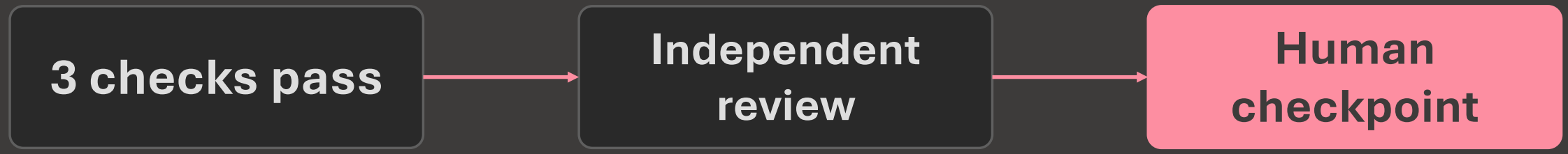
200 seats over capacity.
The candidate must adapt.

A failed check changes the graph.



Same process. A different path.

Passing checks is not permission.




```
await ctx.pause("approve-local-draft");
```

A resumable pause is a feature.
A cancelled run is not a resumable pause.

2,500 attendees accounted for.

2,300

confirmed

\+

200

waitlisted

0 duplicate outcomes	**0 seats over capacity**	**0 missing attendees**

Measured local fixture. Not a capacity or latency benchmark.

Three demo modes.
One honest story.

Offline rehearsal	Local JavaScript	No LLM / no sign-in
Native rehearsal	Copilot runtime	No LLM / preview access
Native live	Copilot agents	Model calls / AI credits

The fallback preserves the lesson—not an illusion of AI.

Autonomy is not authorization.

tools: [] constrains agents.
It does not sandbox extension code.

Review trusted code

Scope permissions

Gate consequences

No generated code execution. No deployment. No remote writes.

Make the workflow a team asset.

`.github\extensions\launch-rescue\extension.mjs`

Reusable process

`.github\agents\launch-analyst.agent.md`

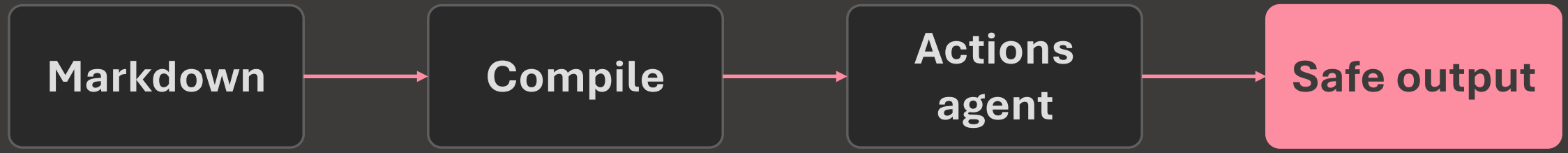
Scoped expertise

`.github\github-app.yml`

Project launch experience

Share the definition. Review the trust. Rehearse the run.

On GitHub Actions, use the other workflow engine.



`.md → .lock.yml`

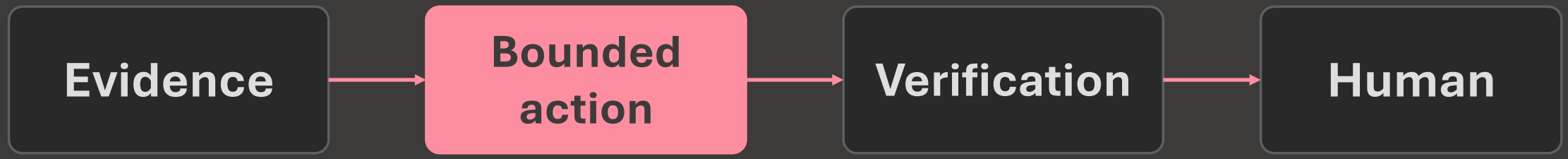
Keep builds and tests deterministic.
Use agents for investigation and interpretation.

Start with one valuable process.

- **A real input** — A failing test, changed files, or incident evidence.
- **A real contract** — What must be true before the work can progress?
- **A judgment step** — Where does interpretation improve the outcome?
- **A human boundary** — What requires review, pause, or approval?

One process, proven end to end, beats ten busy agents.

Give agents a goal.
Give workflows a contract.



Own the process.
Let the path adapt.

PowerPoint + source + offline fallback + native extension included.